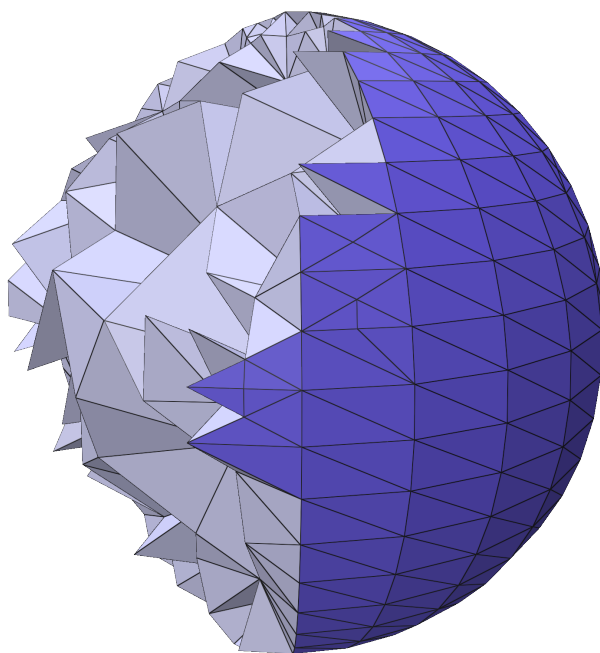


Constrained Delaunay tetrahedrization refinement

Internship report

Max Royer



Supervised by Marco Attene and Marco Livesu

ENS of Lyon
September 2025

Contents

1	Introduction	2
2	Preliminary	3
2.1	Piecewise Linear Complex	3
2.2	Delaunay's triangulation	3
2.3	Constrained Delaunay tetrahedrisation	5
2.4	Quality Measure	5
3	The algorithm	6
3.1	General principle	6
3.2	Local operations	7
3.2.1	Edge split	7
3.2.2	Edge collapse	8
3.2.3	Face swap	9
3.2.4	Edge swap	9
3.2.5	Vertex smoothing	10
3.2.6	Tetrahedron collapse	11
4	Experiments	12
4.1	Source code	12
4.2	One example	12
4.3	Comparison with <i>TetGen</i>	14
5	Conclusion	14

1 Introduction

This report is here to wrap up my internship at the *CNR IMATI* in Genova, Italia. It was a pleasure to share those three months with Marco, Marco and the team (where there is 3 others Marco)!

My internship domain was computational geometry, the domain both Marco Attene and Marco Livesu are working on. Recently Lorenzo Diazzi, Marco A. et al published an article [1] talking about constrained Delaunay tetrahedrization. So far they had produced the robust tetrahedrization pipeline, using robust geometric predicates developped by Marco Attene. But they still lacked of a refinement process for the tetrahedrized output that other known software like TetGen [3], or CGAL [4].

Constrained Delaunay tetrahedrization (CDT) have a wide range of applications, they are commonly used in physics-based simulations such as the finite element method, solutions of partial differential equations or raytracing. However, these simulations and applications require high-quality meshes, which raw CDTs often fail to provide. Therefore, a refinement process is essential to improve the overall mesh quality and ensure the robustness and accuracy of the simulations.

The objective of this internship was to design and validate a greedy algorithm for refining Constrained Delaunay Tetrahedrizations (CDTs). We majorly gathered inspiration from the article [2] that was already proposing a refinement process but in their case they were authorizing themselves to slightly deform the input surface. However in our case the constraints are strict, which means that we cannot remove a input vertex or change the geometry of the faces. We are only allowed to add vertices, which may lay on the surface of the model only if they do not change the original surface geometry.

The plan of this report is to first present some preliminaries and the state of the art, then to present the refinement process and finally to compare it with other ones that already exists.

State of the art

The state of the art is composed for us of two piece of work:

TetGen [3] is a software which was published in 2015 by Hang Si and which is still nowadays a reference regarding constrained Delaunay tetrahedrization. Similarly to our project TetGen has a refinement process at the end of their pipeline where they apply local operations. The thing is that TetGen is not so robust and often crashes or does not converge and thus never finish. One of the objective of our algorithm is then to be robust.

Tetrahedral meshing in the wild (TetWild) [2] is an article which was published in 2018 by Hu et al. in which they introduce the Dirichlet energy that we are using too in the project and a first set of local operations. For their case they were allowing themselves to slightly deform the geometry of the input, in this way they achieved to "correct" some intrinsic malformation of the input (*e.g.* the model has a very sharp needle). This is something we want to avoid and thus one of our other objective is to be strict on the boundary and not allow any deformation. This may lead to have some elements that cannot be refined.

On the other hand *CDT* software published by Lorenzo Diazzi, Marco A. et al [1] is robust and exact on the boundaries but produce low quality meshes whereas *TetGen* and *TetWild* produce high-quality meshes. The goal is to offer a refinement process that is still exact and robust to produce high-quality meshes for *CDT*.

2 Preliminary

2.1 Piecewise Linear Complex

In three dimensions, piecewise linear complexes are extremely flexible representations that allow to model not only 3D surfaces, but also more generic collections of 2D "constraints" that we may want to keep in the final 3D mesh.

Definition 2.1 (Piecewise Linear Complex). A Piecewise Linear Complex (PLC) X is a finite set of vertices, segments and facets of $\mathbb{R}^3$. A facet f is a polygonal region that can have holes and any number of sides, and so is not necessarily convex. Then X must respect those two rules:

- X must be closed by intersection, it means that when intersecting two elements of X the intersection is either empty or is an element of X .
- Every facet of X must be planar.

Here is an example of PLC in figure 1.

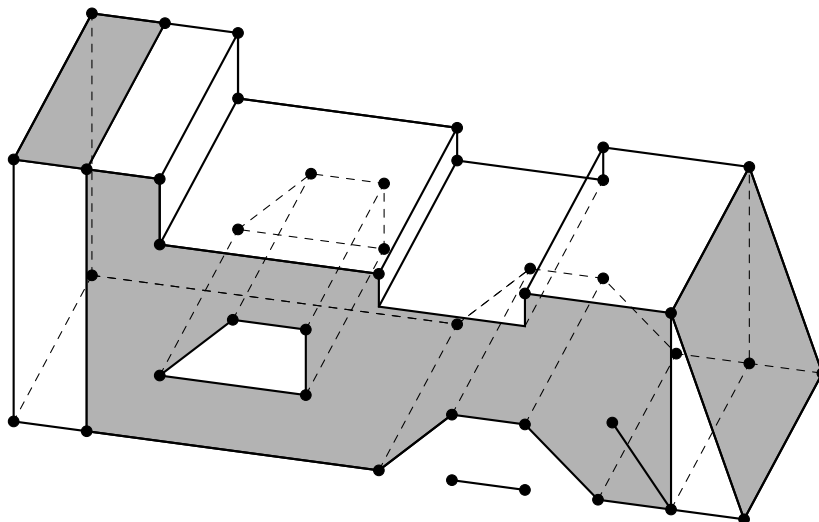


Figure 1: Example of PLC, courtesy of Hang Si

2.2 Delaunay's triangulation

Definition 2.2 (Simplex). A simplex σ of dimension d is the convex hull of $(v_0, v_1, \dots, v_d)$ points of $\mathbb{R}^n$, such that $(v_1 - v_0, \dots, v_d - v_0)$ are linearly independant. The v_i 's are called the vertices of σ , and we say that σ is spanned by $V := \{v_0, \dots, v_d\}$. A face of σ is a simplex spanned by a subset of V , we denote the fact that τ is a face of σ by writing $\tau \leq \sigma$.

Remark 2.1. In dimension 1 a simplex is an segment, in dimension 2 it's a triangle and in dimension 3 it's a tetrahedron.

Definition 2.3 (Simplicial complex). Let K be a set of simplices, we say that K is a simplicial complex if:

- If $\sigma \in K$ then for all τ such that $\tau \leq \sigma$ we have $\tau \in K$.
- Let $\sigma, \sigma' \in K$ then $\sigma \cap \sigma'$ is either empty or a face of both.

Definition 2.4. Let K be a simplicial complex, and let $B \subseteq K$:

- The *closure* of B is :

$$\overline{B} := \{\tau \in K \mid \exists \sigma \in B, \tau \leq \sigma\}$$

- The *star* of B is :

$$\text{St}(B) := \{\sigma \in K \mid \exists \tau \in B, \tau \leq \sigma\}$$

- The *link* of B is :

$$\text{Lk}(B) := \overline{\text{St}(B)} \setminus \text{St}(B)$$

- We also define for simplicity the *vertices* of B that are :

$$\text{Vert}(B) := \{v \in B \mid \dim v = 0\}$$

and the *edges* of B which are :

$$\text{Edges}(B) := \{e \in B \mid \dim e = 1\}$$

Definition 2.5. Some additional notation may be useful in the following, let K be a simplicial complex:

- We define the incident Vertex Tetrahedra's of $v \in K$ where $\dim v = 1$ to be:

$$\text{VT}(v) := \left\{ t \in K \mid \dim t = 3 \text{ and } v \in \overline{\{t\}} \right\}$$

- We define the incident Edge Tetrahedra's of $e \in K$ where $\dim e = 2$ to be:

$$\text{ET}(e) := \left\{ t \in K \mid \dim t = 3 \text{ and } e \in \overline{\{t\}} \right\}$$

Definition 2.6 (Delaunay triangulation). For $P \subseteq \mathbb{R}^d$, a valid Delaunay triangulation is a simplicial complex $DT(P)$ such that for any $\sigma \in DT(P)$, the interior of the *circum-hypersphere* of σ is empty. The later condition is often denoted as the Delaunay condition.

Remark 2.2. In our case we'll only focus on dimension 3, in this case a Delaunay tetrahedrisation exists and is guaranteed to be unique if and only if the points are non coplanar and if no 5 points are cospherical.

2.3 Constrained Delaunay tetrahedrisation

While a Delaunay triangulation provides a structure over a set of points, a "constrained" Delaunay triangulation provides a structure over a set of points and a set of "constraints". In 2D, constraints are segments connecting pairs of the input points. In 3D, constraints can be either segments and/or polygons that, together with the vertices, form a PLC.

Definition 2.7 (Constrained Delaunay tetrahedrisation). Let $X = (V, E, F)$ be a PLC where V is the set of vertices, E is the set of segments, and F is the set of polygonal facets. A constrained Delaunay tetrahedrization C of X is a 3D simplicial complex subdividing the convex hull of X such that:

- The vertex set V' of C contains V
- Any triangular face of C either is part of a facet in F or is locally Delaunay

2.4 Quality Measure

To measure the quality of our refinement we need a quality measure. A good quality measure should be minimized on *well-shaped* tetrahedras and becomes really big on *bad-shaped* tetrahedras. Typically a *well-shaped* tetrahedron would be one close to be a regular tetrahedron, whereas the *bad-shaped* ones are typically slivers which are very flat tetrahedras that appears in constrained and unconstrained Delaunay tetrahedrisation (see on figure 2) or the ones with very acute angle.

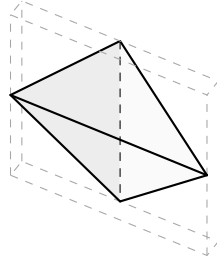


Figure 2: Example of sliver

In our implementation we choosed to use the Dirichlet energy because it's assigning 3 to a regular tetrahedron, is not upper bounded and assigns very high value to slivers.

Definition 2.8 (Edge matrix). Let t be a tetrahedron composed of vertices $(\mathbf{x}_i)_{0 \leq i \leq 3} \in \mathbb{R}^3$, the edge matrix of t is defined to be:

$$T := \begin{pmatrix} \mathbf{x}_1 - \mathbf{x}_0 & \mathbf{x}_2 - \mathbf{x}_0 & \mathbf{x}_3 - \mathbf{x}_0 \end{pmatrix}$$

Remark 2.3. This definition of the edge matrix is invariant by translation.

Definition 2.9 (Tetrahedron deformation). Let t be a tetrahedron and r another one that you want to deform t to, let T and respectively R be the associated edge matrix. We define the deformation matrix to be:

$$D := TR^{-1}$$

Definition 2.10 (Dirichlet energy). Let t be a tetrahedron and J be the deformation matrix when we deform t to the regular tetrahedron of side 1. We define the Dirichlet energy to be:

$$\mathcal{E}(t) := \frac{\text{Tr}(JJ^t)}{\det(J)^{\frac{2}{3}}}$$

Remark 2.4. The definition of this energy makes it so that it's invariant by rotation of t and of the regular tetrahedron chosen, this is why we say *the* regular tetrahedron of side 1.

Remark 2.5. One may wonder if the order of the vertices in the input tetrahedron t is important, the answer is that as soon as the tetrahedron is well-oriented the order does not change the result. If the tetrahedron is badly-oriented the determinant at the denominator will be negative and so we just return the maximum value because it's out of definition for $\frac{2}{3}$ exponent.

3 The algorithm

3.1 General principle

The algorithm is composed of several local operations that I will detail in the following. The main function is the global optimization pass, which means it makes all the different local operations pass in a row. Then one can choose to only run this optimization pass a fixed number of times or to let it run until convergence (until no operations can be applied on the mesh).

The process for each individual operation pass is quite the same for all of them see on figure 3.

Firstly there is the simulation pass, this pass is crucial to know which element on the mesh we will apply the operation. Simulate the effect means to compute the maximum quality measure locally before and after the hypothetic operation on a specific element. There are several step to validate whether the element is good to operate on, here they are:

- The maximum local quality measure has to decrease
- The geometry should not be broken (*e.g.* some tetrahedra may be inverted and then have some negative volume, not deform the boundary of the mesh)
- The topology should be preserved (*e.g.* the manifold structure should be preserved)

The simulation has to take into account all those conditions, and if an element checks every one of them, it is then added in a queue of element to process.

Then after computing the queue, the process is greedy. We pop an element and as it is in the queue we know it is a good element to operate on. So we apply the local operation and retrieve both removed and impacted elements by the operation. Those element are then respectively removed from the queue and resimulated to whether add them in the queue or remove them. We always want to maintain the invariant that if an element is in the queue it is good to operate on.

The difficult part are often to know if the operation will break the topology, and to know which element are impacted by a local operation.

In the following I will present every operation, detailing impacted elements, topological condition to check and what the operation does.

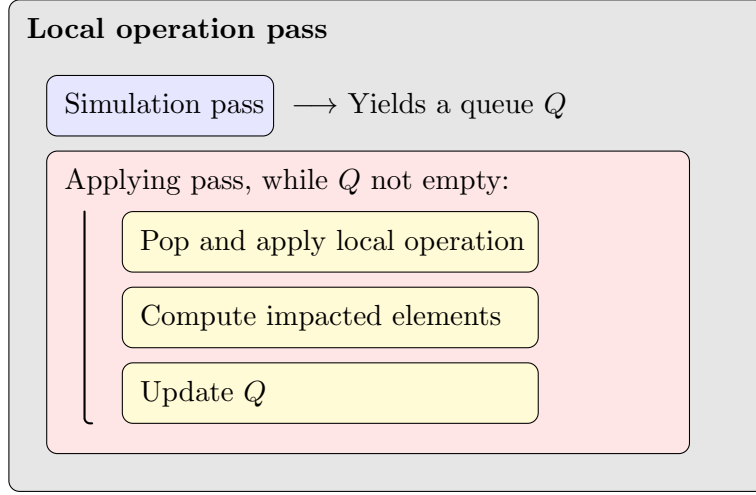


Figure 3: Local operation scheme

3.2 Local operations

3.2.1 Edge split

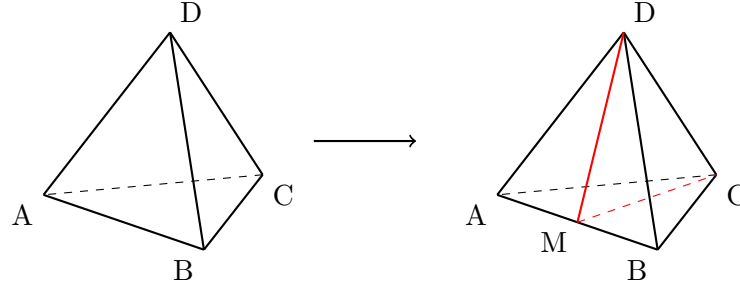


Figure 4: Example of splitting an edge

The edge split as the name suggests is the fact to add a new vertex to an edge and then arrange the geometry accordingly. For this operation we do not have to worry about the fact that it could break the topology because it cannot.

By defining $\mathcal{T} := \text{ET}((u, v))$ the incident tetrahedra of the edge (u, v) . We have that when we split (u, v) by adding w in the middle of it, each $T \in \mathcal{T}$ will be transformed in the following manner:

$$T := \{u, v, x, y\} \longrightarrow T' := \{u, w, x, y\} \text{ and } T'' := \{w, v, x, y\}$$

Let's define $\mathcal{T}' := \{T' \mid T \in \mathcal{T}\} \cup \{T'' \mid T \in \mathcal{T}\}$ the set of transformed tetrahedra. Therefore the operation is valid on the edge (u, v) if:

$$\max_{T' \in \mathcal{T}'} \mathcal{E}(T') \leq \max_{T \in \mathcal{T}} \mathcal{E}(T)$$

Then after applying the operation the edges to update are:

$$E := \text{Edges}(\text{ET}((u, w)) \cup \text{ET}((w, v)))$$

Which are the edges of all the tetrahedra incident to either (u, w) or (w, v) .
Here is an example of an edge split in the figure 4.

3.2.2 Edge collapse

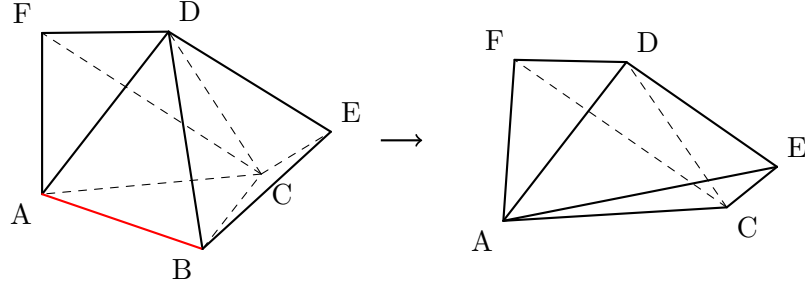


Figure 5: Example of collapsing an edge

Rather the intuitive approach to the edge collapse depicted on figure 5, this is one of the hardest operation to approach. In fact the difficulty of this operation is to prevent topological break. Firstly the vertex or the edge that we are going to remove should not be on the boundary of the model. Then the principal condition to ensure to not break the topology is the link condition for collapsing (u, v) :

$$\text{Lk}(\{(u, v)\}) = \text{Lk}(\{u\}) \cap \text{Lk}(\{v\}) \quad (\text{Link condition})$$

Then when collapsing the edge (u, v) on u the incident tetrahedra at the edge (u, v) $\text{ET}((u, v))$ will be removed whereas each $T \in \text{VT}(v) \setminus \text{ET}((u, v))$ will be transformed in the following manner:

$$T := \{v, a, b, c\} \longrightarrow T' := \{u, a, b, c\}$$

Therefore the operation is valid if it respects the Link condition and if:

$$\max_{T \in \text{VT}(v) \setminus \text{ET}((u, v))} \mathcal{E}(T') \leq \max_{T \in \text{VT}(v)} \mathcal{E}(T)$$

In a second time when collapsing the edge (u, v) on u the removed edges that has to be suppressed from the queue are:

$$R := \text{Edges}(\text{St}(\{v\}))$$

and the edges to update are:

$$E := \text{Edges}(\overline{\text{Vert}(\{v\})}) \setminus R$$

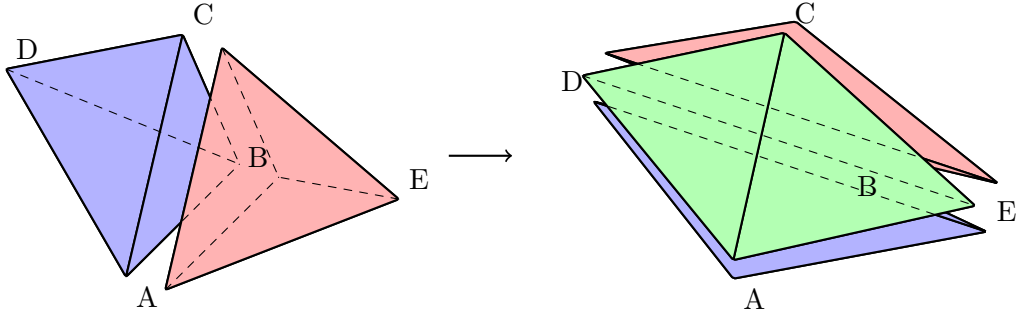


Figure 6: Example of swapping a face

3.2.3 Face swap

The face swap modifies the configuration of two tetrahedra that share a common face $\sigma = (u, v, w)$.

The face swap is defined on two tetrahedra that share a face σ . Denote those tetrahedra as:

$$T_1 = \{u, v, w, a\} \text{ and } T_2 = \{u, v, w, b\}$$

where a and b are the vertices opposite to the shared face σ in each tetrahedron.

The face swap replaces T_1 and T_2 by three new tetrahedra:

$$T'_1 = \{a, b, u, v\}, \quad T'_2 = \{a, b, v, w\}, \quad T'_3 = \{a, b, w, u\}$$

These tetrahedra share the edge (a, b) and together fill the same volume as $T_1 \cup T_2$, ensuring the local topology remains consistent. The operation is then feasible if:

$$\max_{T \in \{T'_1, T'_2, T'_3\}} \mathcal{E}(T) \leq \max_{T \in \{T_1, T_2\}} \mathcal{E}(T)$$

An example of such a face swap operation is illustrated in figure 6.

The faces to update are those of the three new tetrahedras added, which is:

$$F := \left\{ \sigma \in \overline{\{T'_1, T'_2, T'_3\}} \mid \dim \sigma = 2 \right\}$$

3.2.4 Edge swap

The edge swap changes the connectivity around an edge (u, v) shared by multiple tetrahedra. It consists of two steps:

1. Split the edge (u, v) by inserting a new vertex m at its midpoint
2. Collapse m onto a neighbor vertex w distinct from u and v

Let $\mathcal{T} := \text{ET}((u, v))$, when collapsing on vertex w , each tetrahedron $T := \{u, v, a, b\}$ that does not contain w is subdivided into two tetrahedra:

$$T \rightarrow T' := \{u, w, a, b\} \quad \text{and} \quad T'' := \{v, w, a, b\}$$

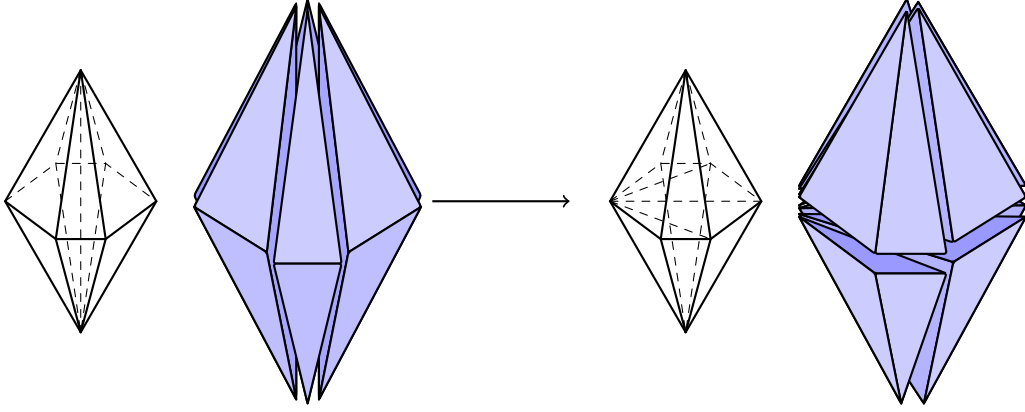


Figure 7: Example of swapping an edge

Whereas the tetrahedron that contains the vertex w is removed. Let's define the updated tetrahedra when collapsing m on w :

$$\mathcal{T}_w := \{T' \mid T \in \mathcal{T}, w \notin T\} \cup \{T'' \mid T \in \mathcal{T}, w \notin T\}$$

Then concerning the topological point of view we still have to check the link condition when collapsing m on w , we can then define the collapsible vertices to be:

$$\mathcal{C} := \{w \in \text{Vert}(\text{Lk}((u, v))) \mid \text{Lk}(\{(m, w)\}) = \text{Lk}(\{m\}) \cap \text{Lk}(\{w\})\}$$

The operation is then valid if:

$$\min_{w \in \mathcal{C}} \max_{T \in \mathcal{T}_w} \mathcal{E}(T) \leq \max_{T \in \text{ET}((u, v))} \mathcal{E}(T)$$

Finally there is no edge that is removed except the one we operated on and the edges to update are:

$$E := \bigcup_{x \in \text{Vert}(\mathcal{T})} \{(x, y) \mid y \text{ is a neighbour of } x\}$$

There is an example in figure 7.

3.2.5 Vertex smoothing

Vertex smoothing is an operation that does not change the connectivity of the mesh. Instead it changes the geometry, by moving vertices to the barycenter of their neighbour. Let v be a vertex and $V := \text{Vert}(\bar{v}) \setminus \{v\}$ the neighbour of v . Then v is moved to:

$$v' := \frac{1}{n} \sum_{u \in V} u$$

And so each $T := \{v, x, y, z\} \in \text{VT}(v)$ is transformed to $T' := \{v', x, y, z\}$ and the operation is valid if it does not invert any tetrahedron and if:

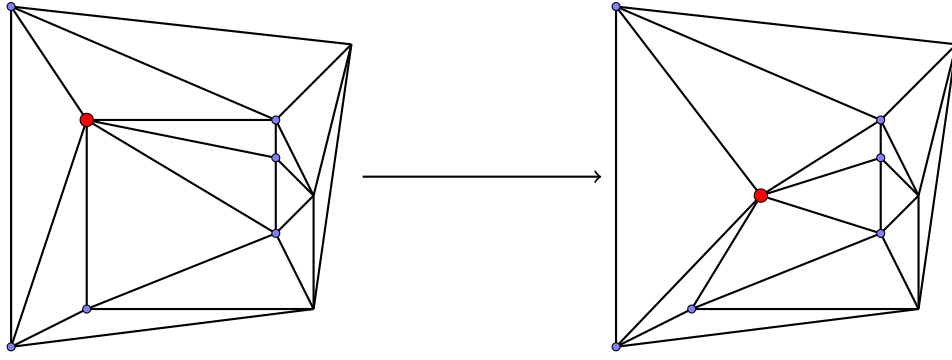


Figure 8: Example of smoothing a vertex

$$\max_{T \in \text{VT}(v)} \mathcal{E}(T') \leq \max_{T \in \text{VT}(v)} \mathcal{E}(T)$$

No vertices have been removed and thus the vertices to update are the ones in V . Here is an example of smoothing a vertex in figure 8 (it is 2D here for simplicity).

3.2.6 Tetrahedron collapse

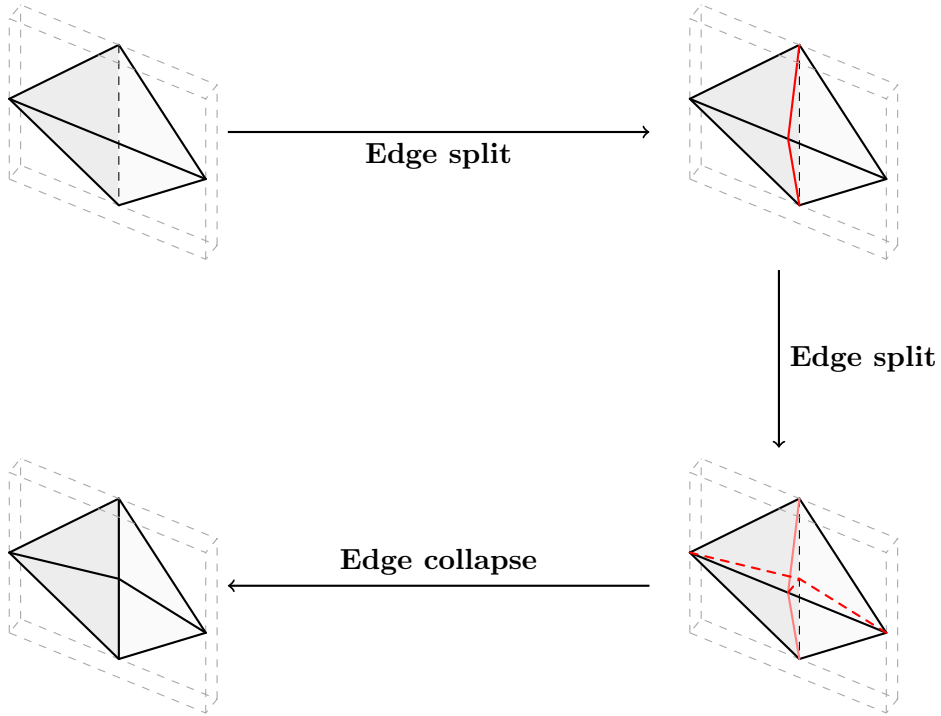


Figure 9: Tetrahedra collapsing process

Slivers as depicted in figure 2 can be hard to get rid of. In fact in certain cases, due to the fact that we are only doing operations if they decrease the maximum quality measures locally, it is hard to remove slivers in one single operation. This is why creating a new combination of already existing operations can help to prevent slivers to appear in the final mesh.

A typical example of slivers that were hard to remove were the one near the boundary. Every single operation would cause to degrade the mesh locally. But seeing further than one operation is possible, and this is what the tetrahedron collapse does.

The tetrahedron collapse on $T := \{u, v, w, x\}$ is a series of three operation, which are:

1. An edge split on (u, v) by adding m
2. An edge split on (w, x) by adding m'
3. An edge collapse of m' on m

By defining $\mathcal{R} := \text{ET}((u, v)) \setminus \{T\}$ and $\mathcal{S} := \text{ET}((w, x)) \setminus \{T\}$ and then define the updated tetrahedra by splitting and collapsing we can define $\mathcal{R}'$ and $\mathcal{S}'$:

$$\text{For } R := \{u, v, a, b\} \in \mathcal{R}, R \longrightarrow R' := \{u, m, a, b\} \in \mathcal{R}' \text{ and } R'' := \{v, m, a, b\} \in \mathcal{R}'$$

$$\text{For } S := \{w, x, a, b\} \in \mathcal{S}, S \longrightarrow S' := \{w, m, a, b\} \in \mathcal{S}' \text{ and } S'' := \{x, m, a, b\} \in \mathcal{S}'$$

Then the operation is valid if:

$$\max_{T \in \mathcal{R}' \cup \mathcal{S}'} \mathcal{E}(T) \leq \max_{T \in \mathcal{R} \cup \mathcal{S} \cup \{T\}} \mathcal{E}(T)$$

The link condition also needs to be checked when collapsing the edge (m, m') .

Finally the set of tetrahedron to update is $\mathcal{R}' \cup \mathcal{S}'$.

4 Experiments

All the experiments I have done were on my machine, a M1 mac with 8 Gb of ram. Here is a link to the github repository where you will be able to reproduce the experiments and run the code.

4.1 Source code

TODO : Maybe briefly present data structures used and organization of the code.

4.2 One example

I will first show some result on one example which is plotted on figure 10. This is quite a complex model with 4557 vertices and 9110 triangles. The CDT process has added 36722 Steiner points and we are starting from a point where the maximum quality measure is $3.81836e + 06$ and the average quality measure is 1599,68. After 49 minutes and 43 seconds our algorithm has converged to a maximum quality measure of 4814,68 and an average of 10,6005. It is respectively a decrease of 79300% and 15000%

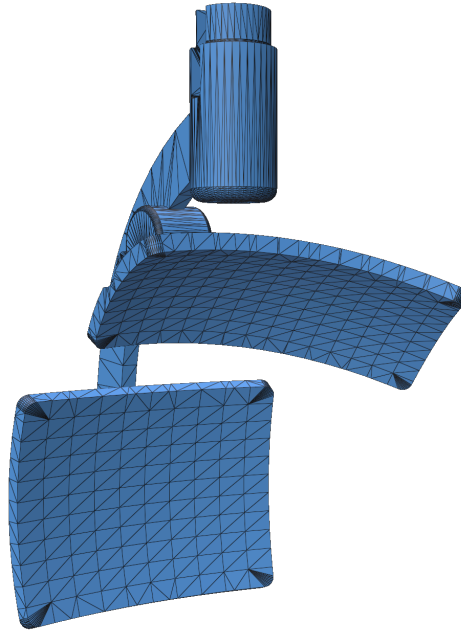


Figure 10: The model 112544

Here is in figure 11 the average energy evolution:

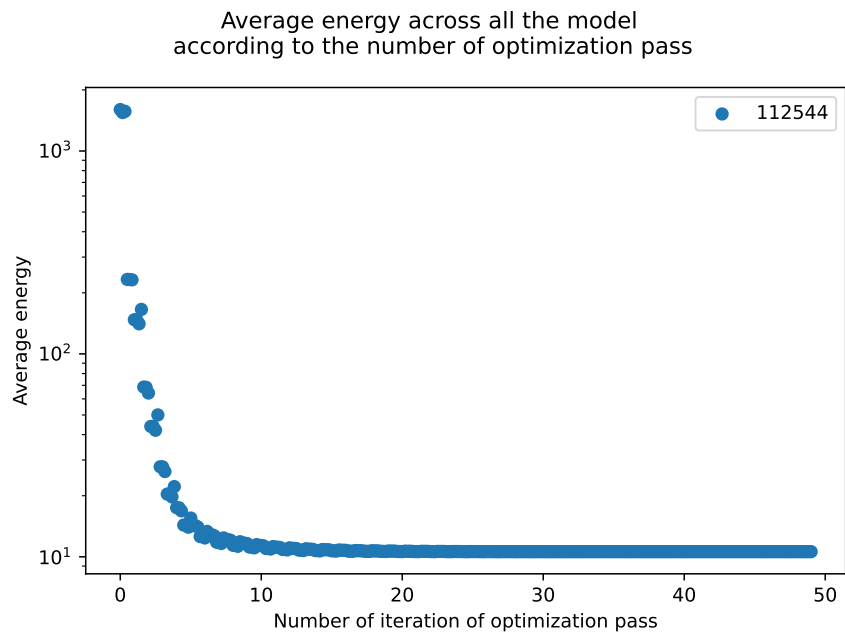


Figure 11: Average energy evolution on model 112544

4.3 Comparison with *TetGen*

TODO

5 Conclusion

TODO

References

- [1] Lorenzo Diazzi et al. *Constrained Delaunay Tetrahedrization: A Robust and Practical Approach*. 2023. arXiv: 2309.09805 [cs.CG]. URL: <https://arxiv.org/abs/2309.09805>.
- [2] Yixin Hu et al. “Tetrahedral meshing in the wild”. In: *ACM Trans. Graph.* 37.4 (July 2018). ISSN: 0730-0301. DOI: 10.1145/3197517.3201353. URL: <https://doi.org/10.1145/3197517.3201353>.
- [3] Hang Si. “TetGen, a Delaunay-Based Quality Tetrahedral Mesh Generator”. In: *ACM Trans. Math. Softw.* 41.2 (Feb. 2015). ISSN: 0098-3500. DOI: 10.1145/2629697. URL: <https://doi.org/10.1145/2629697>.
- [4] The CGAL Project. *CGAL User and Reference Manual*. 6.0.1. CGAL Editorial Board, 2024. URL: <https://doc.cgal.org/6.0.1/Manual/packages.html>.